Postal Service Management

Pseudocode and Algorithm Design

CST2550 | Middlesex University

This Document contains the pseudocode and step-by-step algorithm explanation for all three custom data structure implemented in the Postal Service Management System.

Structure Covered:

1) **Custom Hash Table - O (1) average parcel lookup by tracking ID**
2) **Custom Binary Search Tree (BST) - O (log n) sorted parcel display**
3) **Custom Queue -- O (1) FIFO delivery processing**

All data structures were designed and built from scratch without using any standard
template library (STL) collections, as required by the CST2550 coursework brief.

1. Custom Hash Table

1.1 What is a Hash Table?

A Hash Table is a data structure that stores key-value pairs. It uses a mathematical function called a hash function to convert a key (in our case a parcel tracking ID like PS-2026-00001) into an index number. That index tells the program exactly which slot in an array to look in, so it can find the value instantly without searching through everything.

This gives an average time complexity of O (1) for search, add and delete operations, meaning the time taken does not increase as more parcels are added to the system.

1.2 Why We Used It in Postal MS

When a customer types a tracking ID into the Postal MS system, the application needs to find that parcel as fast as possible. If we stored all parcels in a simple list and searched through them one by one, it would take O(n) time - meaning the more parcels in the system, the slower it gets. The Hash Table solves this by jumping directly to the right slot in O (1) average time, no matter how many parcels exist.

1.3 How Collisions Are Handled

Sometimes two different tracking IDs produce the same index (called a collision). Postal MS handles this using chaining - each slot in the array holds a linked list of nodes. If two keys hash to the same index, both are stored in that slot as a chain, and the search walks through the chain to find the right one.

1.4 How the Hash Function Works

The hash function converts the tracking ID string into a number using polynomial rolling hash. Each character in the string is multiplied by 31 raised to its position power, then the total is taken modulo the table size to get the index. The number 31 is chosen because it is a prime number, which reduces the chance of collisions.

Hash Function Formula:

```
hash = 0
FOR each character c in the key string:
    hash = (hash * 31) + ASCII value of c
index = absolute value of (hash mod table Size)
```

Example: key = 'PS-2026-00001'

Each character is multiplied by 31 and accumulated

Result is taken mod 100 (default table size) to get index between 0 and 99

1.5 Auto-Resize (Load Factor)

When the hash table becomes too full, collisions increase and performance degrades. Postal MS automatically resizes the table when the load factor exceeds 0.75 (meaning more than 75% of slots are used). It doubles the table size and re-inserts all existing items into the new larger table. This resize operation is O (n) but happens rarely.

```
CLASS HashNode:
    key: String     // Tracking ID e.g. PS-2026-00001
    value: Object     // Parcel status e.g. Pending
    next : HashNode    // Pointer to next node (for collision chaining)
```

1.6 Pseudocode

```
Node Structure:

CLASS HashNode:
    key    : String       // Tracking ID e.g. PS-2026-00001
    value  : Object       // Parcel status e.g. Pending
    next   : HashNode     // Pointer to next node in the chain
```

1.6.1 Add (Insert or Update a Parcel)

This operation adds a new parcel tracking ID and status to the hash table. If the key already exists (same tracking ID submitted twice), it updates the value instead of adding a duplicate.

```
FUNCTION Add (key, value):
    // Step 1: Calculate which slot this key belongs in
    index = HashFunction(key)

    // Step 2: Go to that slot and walk the chain
    currentNode = table[index]
    WHILE currentNode is not null:
        // Step 3: If this key already exists, just update the value
        IF currentNode.key equals key:
            currentNode.value = value
            RETURN   // Done - no duplicate created
        currentNode = currentNode.next

    // Step 4: Key not found in chain, so create a new node
    newNode = new Node (key, value)
    // Step 5: Add new node to the FRONT of the chain at this slot
    newNode.next = table[index]
    table[index] = newNode
    count = count + 1
```

```
    // Step 6: Check if table is getting too full (load factor > 0.75)
    IF (count / tableSize) > 0.75:
        CALL Resize () // Double the table size and re-insert all items
END FUNCTION
```

1	**Calculate Index** The hash function converts the tracking ID string into an integer index between 0 and tableSize-1. This tells us exactly which slot to check.
2	**Walk the Chain** Go to the calculated slot and walk through any existing nodes (chain) to check if this key already exists in the table.
3	**Update if Exists** If the tracking ID is already in the table (same parcel submitted again), just update its status value and return immediately without creating a duplicate.
4	**Create New Node** If the key was not found anywhere in the chain, create a brand-new node with this tracking ID and status.
5	**Insert at Front** Add the new node to the front of the chain at this slot. Inserting at the front is O (1) because we do not need to walk to the end.
6	**Check Load Factor** If more than 75% of the table slots are now used, resize the table to avoid performance and degradation from too many collisions.

1.6.2 Search (Find a Parcel by Tracking ID)

This is the most important operation in Postal MS. When a user enters a tracking ID, this function finds its status in O (1) average time by going directly to the correct slot.

```
FUNCTION Search(key):
    // Step 1: Calculate which slot this key would be in
    index = HashFunction(key)

    // Step 2: Go to that slot and walk the chain
    currentNode = table[index]
```

```
    WHILE currentNode is not null:
        // Step 3: Compare each node's key with what we are looking for
        IF currentNode.key equals key:
            RETURN currentNode.value   // Found it - return the status
        currentNode = currentNode.next

    // Step 4: Walked the whole chain and did not find it
    RETURN null   // Parcel does not exist in hash table
END FUNCTION
```

1.6.3 Delete (Remove a Parcel)

Removes a parcel from the hash table when it is deleted from the system. Requires careful pointer management to maintain the chain structure after removal.

```
FUNCTION Delete(key):
    // Step 1: Calculate which slot this key is in
    index = HashFunction(key)

    currentNode = table[index]
    previousNode = null

    WHILE currentNode is not null:
        IF currentNode.key equals key:
            // Step 2: Found the node to delete
            IF previousNode is null:
                // It was the first node in the chain
                table[index] = currentNode.next
            ELSE:
                // Bypass the deleted node in the chain
                previousNode.next = currentNode.next
            count = count - 1
            RETURN true   // Successfully deleted
        previousNode = currentNode
        currentNode = currentNode.next

    RETURN false   // Key was not found
END FUNCTION
```

1.6.4 Resize (Expand the Table)

Called automatically when the load factor exceeds 0.75. Doubles the table size and re-hashes all existing items into the new larger table. This is O(n) but happens rarely.

```
FUNCTION Resize ():
```

```
    // Step 1: Save reference to old table
    oldTable = table
    oldSize = tableSize

    // Step 2: Create new table double the size
    tableSize = tableSize * 2
    table = new empty array of size tableSize
    count = 0

    // Step 3: Re-insert every item from the old table
    // (They get new indices because tableSize changed)
    FOR each slot in oldTable:
        currentNode = oldTable[slot]
        WHILE currentNode is not null:
            CALL Add (currentNode.key, currentNode.value)
            currentNode = currentNode.next
END FUNCTION
```

1.6.5 Clear (Reset the Hash Table)

Removes all items from the hash table and resets the count to zero. Used in PostalMS when the data structures need to be rebuilt from the database on startup. This is O(n) because every slot in the array must be set back to null.

```
FUNCTION Clear():
    // Step 1: Reset every slot in the array to null
    FOR each slot index i from 0 to tableSize - 1:
        table[i] = null

    // Step 2: Reset the item count to zero
    count = 0

    // The table is now completely empty but still the same size
    // ready to accept new items
END FUNCTION
```

1.7 Time Complexity Summary

Operation	Best Case	Average Case	Worst Case	Reason for Worst Case
Add	O (1)	O (1)	O(n)	All keys hash to same slot - full chain walk
Search	O (1)	O (1)	O(n)	Key is at end of a very long chain
Delete	O (1)	O (1)	O(n)	Key is at end of a very long chain
Resize	O(n)	O(n)	O(n)	Must re-hash every existing item
Clear	O(n)	O(n)	O(n)	Must set every array slot to null

2. Custom Binary Search Tree (BST)

2.1 What is a Binary Search Tree?

A Binary Search Tree is a data structure where each node has at most two children - a left child and a right child. The rule is: everything in the left subtree of a node has a smaller key than the node, and everything in the right subtree has a larger key. This ordering property means that searching for a specific key takes O (log n) average time because at each step we eliminate half of the remaining nodes.

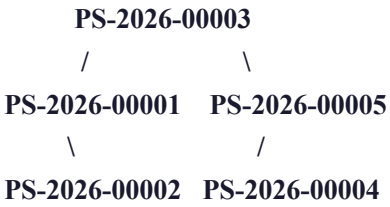
2.2 Why We Used It in Postal MS

Postal MS needs to display parcels in sorted order by tracking ID on the dashboard. If we just stored parcels in a list and sorted them every time the page loaded, it would be O(n log n) every refresh. The BST maintains sorted order automatically as items are inserted. An in-order traversal of the BST visits every node in ascending order by tracking ID in O(n) time, giving us a pre-sorted list without any extra sorting step.

2.3 How Traversal Works

In-order traversal visits nodes in the sequence: Left subtree first, then the Root node, then the Right subtree. Because smaller keys are always in the left subtree and larger keys are always in the right subtree, this sequence produces nodes in ascending alphabetical order. This is used to display the parcel dashboard in sorted order.

In-Order Traversal Example:



In-order visits: 00001, 00002, 00003, 00004, 00005 (ascending order)

This is how Postal MS displays parcels sorted by tracking ID on the dashboard.

```
CLASS BSTNode:
  key   : String     // Tracking ID
  value : Object     // Parcel status
  left  : BSTNode    // Left child (smaller keys)
  right : BSTNode    // Right child (larger keys)
```

2.4 Pseudocode

```
Node Structure:

CLASS BSTNode:
    key   : String      // Tracking ID
    value : Object      // Parcel status
    left  : BSTNode     // Left child — smaller keys
    right : BSTNode     // Right child — larger keys
```

2.4.1 Insert (Add a Parcel to the BST)

Inserts a new parcel into the correct position in the BST based on its tracking ID. The tree structure ensures all insertions maintain the sorted order property automatically.

```
FUNCTION Insert(key, value):
    root = InsertRecursive(root, key, value)
END FUNCTION

FUNCTION InsertRecursive(node, key, value):
    // Step 1: If we have reached an empty spot, create the new node here
    IF node is null:
        count = count + 1
        RETURN new Node(key, value)

    // Step 2: Decide which direction to go
    IF key is less than node.key:
        // Step 3a: Key is smaller - go LEFT (smaller values live on the left)
        node.left = InsertRecursive(node.left, key, value)

    ELSE IF key is greater than node.key:
        // Step 3b: Key is larger - go RIGHT (larger values live on the right)
        node.right = InsertRecursive(node.right, key, value)

    ELSE:
        // Step 3c: Key already exists - just update the value (no duplicate)
        node.value = value

    // Step 4: Return the node back up the recursive call chain
    RETURN node
END FUNCTION
```

1	**Empty Spot Found** When the recursive function reaches a null pointer, it has found the correct empty position for the new node. A new node is created here and the count is incremented.

2	**Compare Keys** At each node, compare the new key with the current node's key to decide whether to go left (smaller) or right (larger).

3a	**Go Left** If the new key is smaller than the current node, the new node belongs somewhere in the left subtree. Recurse left.
3b	**Go Right** If the new key is larger than the current node, the new node belongs somewhere in the right subtree. Recurse right.
3c	**Update Duplicate** If the key already exists in the tree (same tracking ID), update its value instead of creating a duplicate node.
4	**Return Node** Each recursive call returns the node back up the chain, so the parent's left or right pointer is correctly linked to the updated subtree.

2.4.2 Search (Find a Parcel in the BST)

Searches for a parcel by tracking ID using the BST's ordering property. At each node, it compares the target key and eliminates half the remaining tree, giving O (log n) average performance.

```
FUNCTION Search(key):
    result = SearchRecursive(root, key)
    IF result is null:
        RETURN null   // Not found
    RETURN result.value
END FUNCTION

FUNCTION SearchRecursive(node, key):
    // Step 1: Reached end of tree - key does not exist
    IF node is null:
        RETURN null

    // Step 2: Compare the target key with current node
    IF key is less than node.key:
        // Step 3a: Target is smaller - it must be in the LEFT subtree
        RETURN SearchRecursive(node.left, key)

    ELSE IF key is greater than node.key:
        // Step 3b: Target is larger - it must be in the RIGHT subtree
        RETURN SearchRecursive(node.right, key)
```

```
    ELSE:
        // Step 4: Keys match - this is the node we are looking for
        RETURN node
END FUNCTION
```

2.4.3 In-Order Traversal (Get All Parcels Sorted)

Visits every node in the BST in ascending order by tracking ID. This is the core operation used to display the sorted parcel list on the Postal MS dashboard. It always visits left child first, then the current node, then right child.

```
FUNCTION InOrder():
    result = empty list
    InOrderRecursive(root, result)
    RETURN result   // Contains all parcels sorted by tracking ID
END FUNCTION

FUNCTION InOrderRecursive(node, result):
    // Base case: if node is null, there is nothing to visit
    IF node is null:
        RETURN

    // Step 1: Visit everything in the LEFT subtree first
    // (Left subtree contains all smaller keys)
    InOrderRecursive(node.left, result)

    // Step 2: Visit the current node
    // (At this point, all smaller keys have already been added)
    result.Add(node.key, node.value)

    // Step 3: Visit everything in the RIGHT subtree
    // (Right subtree contains all larger keys)
    InOrderRecursive(node.right, result)
END FUNCTION
```

2.4.4 Find Minimum (Get Earliest Parcel)

Finds the parcel with the smallest tracking ID in the BST. Because the smallest key is always in the leftmost node, this simply follows left child pointers until it reaches a node with no left child.

```
FUNCTION FindMin():
    // Cannot find minimum of an empty tree
    IF root is null:
        RETURN null
```

```
    currentNode = root

    // Step 1: Keep going LEFT until there is no more left child
    // The leftmost node has the smallest key in the entire tree
    WHILE currentNode.left is not null:
        currentNode = currentNode.left

    // Step 2: currentNode is now the leftmost node = minimum key
    RETURN currentNode
END FUNCTION
```

2.4.5 Delete (Remove a Parcel from BST)

Removing a node from a BST is the most complex operation because the tree structure must be maintained after the removal. There are three cases to handle depending on how many children the node has.

Three cases for deletion:

Case 1 - Node has NO children (leaf): Simply remove it, set parent pointer to null.
Case 2 - Node has ONE child: Replace the node with its only child.
Case 3 - Node has TWO children: Replace with the in-order successor (smallest node in the right subtree), then delete the in-order successor from its original position. This maintains the BST ordering property.

```
FUNCTION Delete(key):
    IF Contains(key) is false:
        RETURN false   // Key does not exist, nothing to delete
    root = DeleteRecursive(root, key)
    count = count - 1
    RETURN true
END FUNCTION

FUNCTION DeleteRecursive(node, key):
    IF node is null:
        RETURN null

    IF key is less than node.key:
        // Target is in the left subtree
        node.left = DeleteRecursive(node.left, key)

    ELSE IF key is greater than node.key:
```

```
        // Target is in the right subtree
        node.right = DeleteRecursive(node.right, key)

    ELSE:
        // Found the node to delete - handle the three cases

        // Case 1 and 2: No left child - replace with right child (may be null)
        IF node.left is null:
            RETURN node.right

        // Case 2: No right child - replace with left child
        IF node.right is null:
            RETURN node.left

        // Case 3: Node has two children
        // Find the in-order successor (minimum of right subtree)
        successor = FindMinNode(node.right)
        // Copy successor's data into current node
        node.key   = successor.key
        node.value = successor.value
        // Delete the successor from the right subtree
        node.right = DeleteRecursive(node.right, successor.key)

    RETURN node
END FUNCTION
```

2.4.6 Clear (Reset the BST)

Removes all nodes from the BST by setting the root to null. Because C# uses garbage collection, all nodes that are no longer reachable from the root are automatically cleaned up from memory. This is effectively O(1) for the operation itself, though garbage collection runs in O(n).

```
FUNCTION Clear():
    // Step 1: Set root to null
    // All nodes are now unreachable and will be
    // cleaned up automatically by garbage collection
    root = null

    // Step 2: Reset the node count to zero
    count = 0
```

```
    // The tree is now empty and ready to accept new insertions
END FUNCTION
```

2.5 Time Complexity Summary

Operation	Best Case	Average Case	Worst Case	Reason for Worst Case
Insert	O (log n)	O (log n)	O(n)	Degenerate tree (sorted input creates a linked list)
Search	O (1)	O (log n)	O(n)	Target is at deepest leaf of degenerate tree
Delete	O (log n)	O (log n)	O(n)	Target is at deepest leaf of degenerate tree
In-Order	O(n)	O(n)	O(n)	Must visit every node exactly once
Find Min	O (1)	O (log n)	O(n)	Root has no left child (best) vs full depth (worst)
Clear	O (1)	O (1)	O (1)	Just sets root to null, GC handles the rest

3. Custom Queue

3.1 What is a Queue?

A Queue is a First In First Out (FIFO) data structure. Items are always added to one end (called the rear) and removed from the other end (called the front). This mirrors a real-world queue - the first person to join the queue is the first person to be served. PostalMS uses a Queue implemented as a linked list of nodes, with a front pointer for removal and a rear pointer for insertion.

3.2 Why We Used It in Postal MS

When a parcel is submitted in Postal MS, a delivery is automatically created and added to the pending delivery queue. Deliveries must be processed in the exact order they were assigned - if DEL-001 was assigned before DEL-002, then DEL-001 must be processed first. A Queue naturally enforces this ordering without any sorting. Both Enqueue and Dequeue operate in O (1) constant time regardless of how many deliveries are waiting.

FIFO Example in Postal MS:

Parcel submitted at 10:00 --> DEL-001 added to REAR of queue
Parcel submitted at 10:05 --> DEL-002 added to REAR of queue
Parcel submitted at 10:10 --> DEL-003 added to REAR of queue

Queue now looks like: FRONT [DEL-001 --> DEL-002 --> DEL-003] REAR

First delivery processed: DEL-001 (removed from FRONT)
Second delivery processed: DEL-002
Third delivery processed: DEL-003 (last in, last out)

The delivery assigned first is always processed first.

3.3 Internal Structure

The Queue is implemented as a singly linked list. Each node contains a value (the delivery ID) and a pointer to the next node. The Queue class maintains two pointers: front points to the first node (for dequeue) and rear points to the last node (for enqueue). This design gives O (1) for both operations because we never need to traverse the list.

```
CLASS QueueNode:
    value: Object      // Delivery ID e.g. DEL-001
    next: QueueNode    // Pointer to next node in queue
CLASS CustomQueue:
    front: QueueNode   // Points to first item (for dequeue)
    rear: QueueNode    // Points to last item (for enqueue)
    count: Integer     // Number of items in queue
```

3.4 Pseudocode

```
Node Structure:

CLASS QueueNode:
    value : Object      // Delivery ID e.g. DEL-001
    next  : QueueNode   // Pointer to next node
CLASS CustomQueue:
    front : QueueNode   // Points to first item — for dequeue
    rear  : QueueNode   // Points to last item — for enqueue
    count : Integer     // Number of items currently in queue
```

3.4.1 Enqueue (Add a Delivery to the Queue)

Adds a new delivery ID to the back of the queue. Always operates in O (1) time because the rear pointer gives direct access to the last node without traversal.

```
FUNCTION Enqueue(value):
    // Step 1: Create a new node to hold this delivery ID
    newNode = new Node(value)
    newNode.next = null

    // Step 2: Check if the queue is currently empty
    IF rear is null:
```

```
        // Queue is empty - new node is both front and rear
        front = newNode
        rear  = newNode
    ELSE:
        // Step 3: Queue has items - attach new node to the end
        rear.next = newNode   // Current last node points to new node
        rear = newNode        // Update rear pointer to new last node

    count = count + 1
END FUNCTION
```

1	**Create New Node** A new node is created containing the delivery ID that needs to be queued. Its next pointer is set to null because it will be the last item in the queue.

2	**Empty Queue Check** If the queue is empty (rear is null), the new node is both the first and last item, so both front and rear pointers point to it.

3	**Attach to Rear** If the queue has existing items, the current last node's next pointer is updated to point to the new node. The rear pointer is then moved to the new node. O(1) - no traversal needed.

3.4.2 Dequeue (Remove and Return the Front Delivery)

Removes the delivery at the front of the queue and returns it. This is always the delivery that was waiting the longest. Operates in O (1) time because the front pointer gives direct access.

```
FUNCTION Dequeue ():
    // Step 1: Cannot remove from an empty queue
    IF front is null:
        THROW exception 'Queue is empty - no deliveries to process'

    // Step 2: Save the value from the front node before removing it
    value = front.value

    // Step 3: Move the front pointer to the next node in the chain
```

```
    front = front.next

    // Step 4: If the queue is now empty, reset the rear pointer too
    IF front is null:
        rear = null   // Queue is now completely empty

    count = count - 1
    RETURN value   // Return the delivery ID that was at the front
END FUNCTION
```

1	**Empty Check** If front is null, the queue is empty. Attempting to dequeue from an empty queue throws an InvalidOperationException to alert the caller.
2	**Save Value** Save the delivery ID from the front node before the pointer moves, so we can return it at the end.
3	**Advance Front** Move the front pointer to the next node. The old front node is no longer referenced and will be cleaned up by garbage collection.
4	**Reset if Empty** If advancing the front pointer results in front being null, the queue is now empty. The rear pointer must also be set to null to keep both pointers consistent.

3.4.3 Peek (View Front Without Removing)

Returns the value at the front of the queue without removing it. Useful for checking which delivery would be processed next without processing it. Always O (1).

```
FUNCTION Peek ():
    // Step 1: Cannot peek at an empty queue
    IF front is null:
        THROW exception 'Queue is empty - nothing to peek at'

    // Step 2: Return the front value without changing any pointers
    // (Queue is completely unchanged after this operation)
```

```
    RETURN front.value
END FUNCTION
```

3.4.4 Is Empty (Check if Queue Has Deliveries)

Checks whether the queue currently contains any pending deliveries. Returns true if empty, false if there are items waiting. O (1) - just checks if the front pointer is null.

```
FUNCTION Is Empty():
    // If front is null, there are no nodes in the queue
    RETURN (front is null)
END FUNCTION
```

3.4.5 ToList (View All Pending Deliveries)

Returns a list of all delivery IDs currently in the queue in FIFO order (front to rear), without removing any of them. Used to display the queue contents in the PostalMS Data Structures demo page. O(n) - must visit every node.

```
FUNCTION ToList():
    result = empty list
    currentNode = front   // Start from the front (oldest delivery)

    // Walk through every node from front to rear
    WHILE currentNode is not null:
        result.Add(currentNode.value)
        currentNode = currentNode.next

    // Note: Queue is completely unchanged - no items removed
    RETURN result   // List is in FIFO order (front first, rear last)

END FUNCTION
```

3.4.6 Clear (Reset the Queue)

Empties the queue by setting both the front and rear pointers to null and resetting the count. All nodes become unreachable and are cleaned up by garbage collection. Used in PostalMS when the delivery queue needs to be rebuilt. O (1) for the operation itself

```
FUNCTION Clear ():
    // Step 1: Set front pointer to null
    // This makes all nodes unreachable from the front
    front = null

    // Step 2: Set rear pointer to null
    // Both ends of the queue are now disconnected
    rear = null

    // Step 3: Reset the count to zero
    count = 0

    // Queue is now completely empty
    // ready to accept new deliveries
END FUNCTION
```

3.5 Time Complexity Summary

Operation	Best Case	Average Case	Worst Case	Reason
Enqueue	O (1)	O (1)	O (1)	Rear pointer gives direct access to last node
Dequeue	O (1)	O (1)	O (1)	Front pointer gives direct access to first node
Peek	O (1)	O (1)	O (1)	Just reads the front pointer value

Operation	Best Case	Average Case	Worst Case	Reason
`IsEmpty`	`O (1)`	`O (1)`	`O (1)`	`Just checks if front pointer is null`
`ToList`	`O(n)`	`O(n)`	`O(n)`	`Must traverse every node from front to rear`
`Clear`	`O (1)`	`O (1)`	`O (1)`	`Just sets front and rear pointers to null`

4. Data Structure Comparison and Justification

4.1 Why These Three Structures?

PostalMS uses all three custom data structures simultaneously because each one solves a different problem within the application. They complement each other - the Hash Table provides fast lookup, the BST provides sorted display, and the Queue provides ordered delivery processing.

Structure	Primary Purpose in PostalMS	Key Operation	Why Not an Alternative?
Hash Table	Fast parcel lookup by tracking ID when user enters it in search or AI chat	Search: O (1) average	A sorted array search would be O(log n). A linked list search would be O(n). Hash Table gives the fastest possible lookup.
BST	Display all parcels sorted by tracking ID on the dashboard	In-Order: O(n) already sorted	Sorting an unsorted list each time would be O (n log n). BST maintains sorted order as items insert, so traversal is just O(n).
Queue	Process pending deliveries in the order they were assigned (FIFO)	Enqueue/Dequeue: O (1)	A sorted list would require O(n) insertion to maintain order. Queue gives O (1) for both add

Structure	Primary Purpose in PostalMS	Key Operation	Why Not an Alternative?
			and remove with guaranteed FIFO ordering.

4.2 How They Work Together – Workflow Example

The following is a workflow example showing the sequence of operations that occur internally when a user submits a new parcel. This is not a function definition but a step-by-step trace of how all three data structures and the database are updated simultaneously.

When a user submits a parcel in PostalMS, all three data structures are updated at the same moment:

```
// WORKFLOW EXAMPLE — not a function definition
// Shows internal operations when user submits parcel PS-2026-00008

BEGIN SubmitParcel(trackingID = "PS-2026-00008", status = "Pending"):
    // Step 1: Add to Hash Table for instant O(1) lookup
    HashTable.Add("PS-2026-00008", "Pending")

    // Step 2: Insert into BST for sorted dashboard display
    BST.Insert("PS-2026-00008", "Pending")

    // Step 3: Save permanently to SQL Server database
    Database.AddParcel("PS-2026-00008", ...all parcel details...)

    // Step 4: Auto-assign delivery and add to processing queue
    newDeliveryID = "DEL-00123"
    DeliveryQueue.Enqueue(newDeliveryID)

END SubmitParcel
// After this workflow completes:
// - Hash Table holds PS-2026-00008 for O (1) instant tracking
// - BST holds PS-2026-00008 in sorted position for dashboard display
// - SQL Server holds all parcel details permanently
// - Queue holds DEL-00123 ready to be processed in order
```

This design means that:

- A user tracking a parcel gets an instant O (1) response from the Hash Table
- The dashboard always shows parcels in sorted order from the BST without any sorting overhead
- Deliveries are processed in the correct order from the Queue
- All data is permanently stored in SQL Server for persistence across application restarts